

Phase 5 Addendum — Supervised Baselines Run as Policies In-Simulator

MS thesis: ICS-NAD-Gym + LLM-shaped RL for ICS intrusion detection

Generated 2026-07-09 · Closes a gap in Phase 4: the 10 supervised classifiers had F1 numbers but no reward numbers · Pipeline:

`src/rl/evaluate_classifiers_in_sim.py`

Why this report exists. A direct question surfaced a real gap: Phase 4 evaluated the 10 supervised classifiers as static row-level predictors on held-out samples — F1/precision/recall only, entirely outside `ICSNADGym`. They never took a sequential action, never triggered the quarantine mechanic, never accrued the asymmetric reward. This report wraps each trained classifier as a policy (predict attack → BLOCK, predict benign → ALLOW) and runs it through the exact same simulator episodes used to evaluate the RL models — giving every model a reward number in the same currency, for the first time.

1. Executive Summary

10 / 10

classifiers run as in-sim
policies

2

classifiers scored negative
mean reward

0.017 SE

RandomForest vs RL+LLM
test gap (noise)

3749

best val reward:
RecurrentPPO v2+LLM

The clearest finding: **F1 does not reliably predict reward**. SVC and GaussianNB — both with "reasonable" F1 (~0.77 on test) — scored *negative* mean reward once their misclassifications interacted with the quarantine mechanic's compounding costs. On validation, the RL models (both v2 and v2+LLM) clearly lead every classifier. On test, RandomForest edges out RL+LLM by 11 points (2398.87 vs 2387.77) — but that gap is 0.017 standard errors given a std of 2561 across just 15 episodes, i.e. indistinguishable from noise, not a real difference.

1.1 A performance bug caught along the way

The first implementation called each classifier's `.predict()` on a single row at a time, inside the per-step simulator loop — mirroring how a live deployment would actually work, but ~1024 individual Python-level calls per episode × 30 episodes × 10 classifiers ≈ 300,000 individual predictions. It was still running after **46 minutes of CPU time with zero output** before being killed. Since an episode's flow sequence in `ICSNADGym` doesn't depend on the agent's actions, all rows for an episode can be batch-predicted once, then stepped through — reducing ~300,000 individual calls to ~30 batched ones. The corrected version completes the entire evaluation in well under a minute.

2. Full Results

2.1 Validation split

Model	Reward	Std	Precision	Recall	F1
RecurrentPPO v2+LLM	3749.28	—	0.749	0.880	0.810
RecurrentPPO v2 (RL-only)	3739.47	—	0.755	0.877	0.811
KNN	3295.67	6532.93	0.907	0.834	0.869
ELM	2728.13	5764.59	0.900	0.866	0.883
GaussianNB	2683.93	5479.67	0.954	0.795	0.867
RandomForest	2484.20	4697.29	0.838	0.897	0.866
AdaBoost	2398.27	8886.36	0.899	0.830	0.863
GBDT	1850.73	6901.64	0.746	0.823	0.783
LightGBM	1785.93	3946.47	0.950	0.881	0.914
XGBoost	1441.40	6399.31	0.747	0.794	0.770
ANN	449.93	8340.97	0.966	0.740	0.838
SVC	−952.67	8197.72	0.809	0.597	0.687

2.2 Test split

Model	Reward	Std	Precision	Recall	F1
RandomForest	2398.87	2561.27	0.906	0.967	0.935
RecurrentPPO v2+LLM	2387.77	—	0.982	0.845	0.909
XGBoost	2274.67	2521.05	0.821	0.955	0.883
GBDT	2235.07	4324.95	0.992	0.897	0.942
RecurrentPPO v2 (RL-only)	1969.60	—	0.996	0.734	0.845
AdaBoost	1645.00	3939.80	0.778	0.875	0.824
ANN	1363.07	3075.15	0.886	0.904	0.895
LightGBM	1433.07	3091.77	0.889	0.897	0.893
ELM	877.20	3597.49	0.995	0.778	0.873
KNN	308.87	3552.99	0.937	0.728	0.819
SVC	−98.93	4262.03	0.872	0.692	0.772
GaussianNB	−395.93	4003.28	0.886	0.686	0.773

These F1 values differ from Phase 4's (e.g. GBDT test F1 here is 0.942 vs. 0.985 in Phase 4) because they're computed over a different row distribution — 15 episode-windows sampled with the same 70%-attack-biased policy used for the RL evaluations (seed=123), not Phase 4's independently-drawn stratified sample. This report's numbers are the ones directly comparable to the RL arms; Phase 4's remain the correct reference for the pure supervised-learning generalization question.

3. Why F1 Doesn't Predict Reward Here

None of these 10 classifiers were trained against the asymmetric cost structure — they minimize a standard classification loss (or maximize accuracy/F1), with no notion that a false BLOCK on a benign flow costs -10 immediately plus an ongoing $-3/\text{step}$ penalty for the next 50 flows if that source repeats, while a missed attack costs -20 . A classifier with decent-but-imperfect precision on borderline flows can rack up compounding quarantine penalties in a way a static F1/precision/recall score never surfaces.

SVC and GaussianNB are the clearest cases: $F1 \approx 0.77$ on test looks like a middling-but-workable classifier by ordinary ML standards. Run as an actual sequential policy, both average *negative* reward — meaning, on average, following either of their decisions is *worse than doing nothing* against this specific asymmetric cost function. This is the sharpest empirical evidence yet for the thesis's central methodological argument: classification quality and sequential-decision quality are genuinely different objectives, and the RL framing exists specifically because it optimizes the one that actually matters operationally.

4. Honest Caveats

Small n, large variance. Every model here (RL included) was evaluated on 15 episodes per split, and standard deviations are frequently as large as or larger than the means (e.g. RandomForest test: mean 2398.87, std 2561.27). The RandomForest-vs-RL+LLM "tie" on test (11-point gap, 0.017 standard errors) is well within noise — this is not evidence that a tree ensemble matches RL+LLM's performance, only that this particular sample size can't distinguish them. The RL-vs-classifier val-split gap (3739-3749 vs. next-best 3296) is larger and more likely a real effect, but still warrants the same multi-seed replication flagged as a next step in the main Phase 5 report.

Action mapping is a simplification. Each classifier's binary attack/benign prediction was mapped directly to BLOCK/ALLOW — there's no ALERT-only option for a classifier-as-policy the way the RL agent has. A more nuanced mapping (e.g. using predicted probability to choose ALERT vs BLOCK by confidence) might change these numbers meaningfully, particularly for models like SVC/GaussianNB whose negative reward may partly reflect this all-or-nothing mapping rather than the classifier's raw discriminative quality alone.

5. Files Created

Module	Purpose
<code>train_supervised_baselines.py</code>	(updated) now persists trained model + scaler objects via <code>joblib</code> , not just metrics
<code>evaluate_classifiers_in_sim.py</code>	Batched classifier-as-policy evaluator: runs each trained model through real <code>ICSNADGym</code> episodes

Outputs:

`data/baseline_models/{scaler,GBDT,RandomForest,AdaBoost,LightGBM,XGBoost,ELM,ANN,KNN,SVC,GaussianNB}.joblib`.